

Computer Programming: Lecture 1

Outline

- Introduction to computation
- Basic descriptions of computers
- Introduction to programs
- Introduction to programming languages

Computation

- Until 1940s the word computer was a job title.
- Computers were people trained to carry out mathematical operations
- They followed a set of precise instructions required to create mathematical tables used for navigation, astronomy, business

What is Computation?

- Computation refers to the process of performing mathematical calculations or solving problems using algorithms and data manipulation.
 - An algorithm is a well-defined, step-by-step procedure or set of rules for solving a specific problem or performing a particular task.
- Computation involves the execution of a series of instructions or steps to obtain a desired result or output.

Examples of Computations

Goal: Calculating an area of a triangle

Input: Base and Height

Compute the area: $\frac{1}{2} * \text{Base} * \text{Height}$

Output the area

Goal: Calculating an average grade of a student in 5 subjects

Input: list<grades>

number of subjects->Count number of grades

total:sum(list_grades)

average->total/number of subjects

Output the average

What are programs?

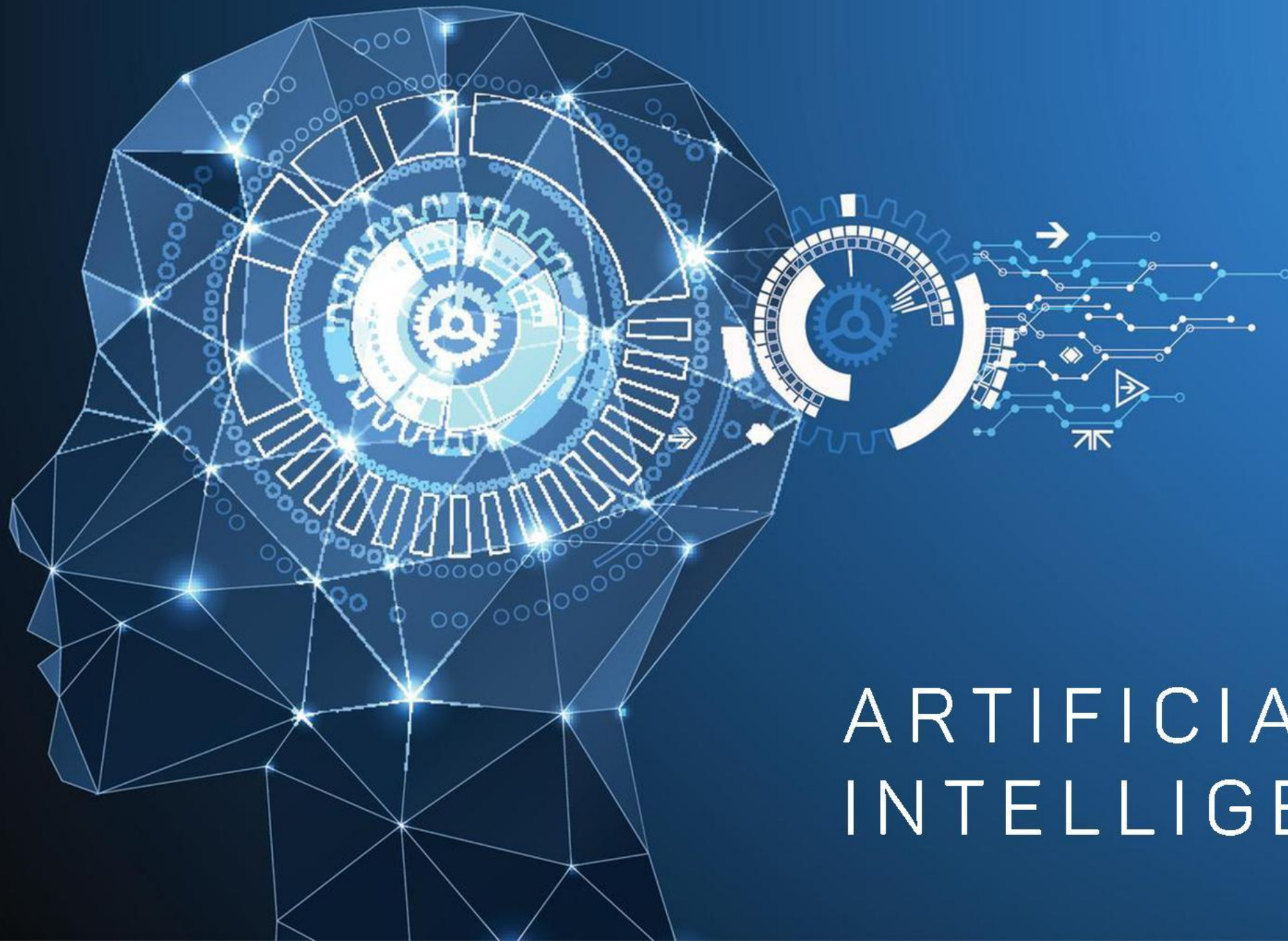
- Computer programs, also known as software, are sequences of instructions that tell a computer what to do.
- They are written in programming languages and can perform a wide range of tasks, from simple calculations to complex simulations and analyses.
- They can be categorized as:
 - System Software: includes operating systems, device drivers, and utility programs that manage and control the computer's hardware and software resources.
 - Application Software: includes programs that perform specific tasks, such as word processing, spreadsheet analysis, and graphic design.

Complex Computation Example



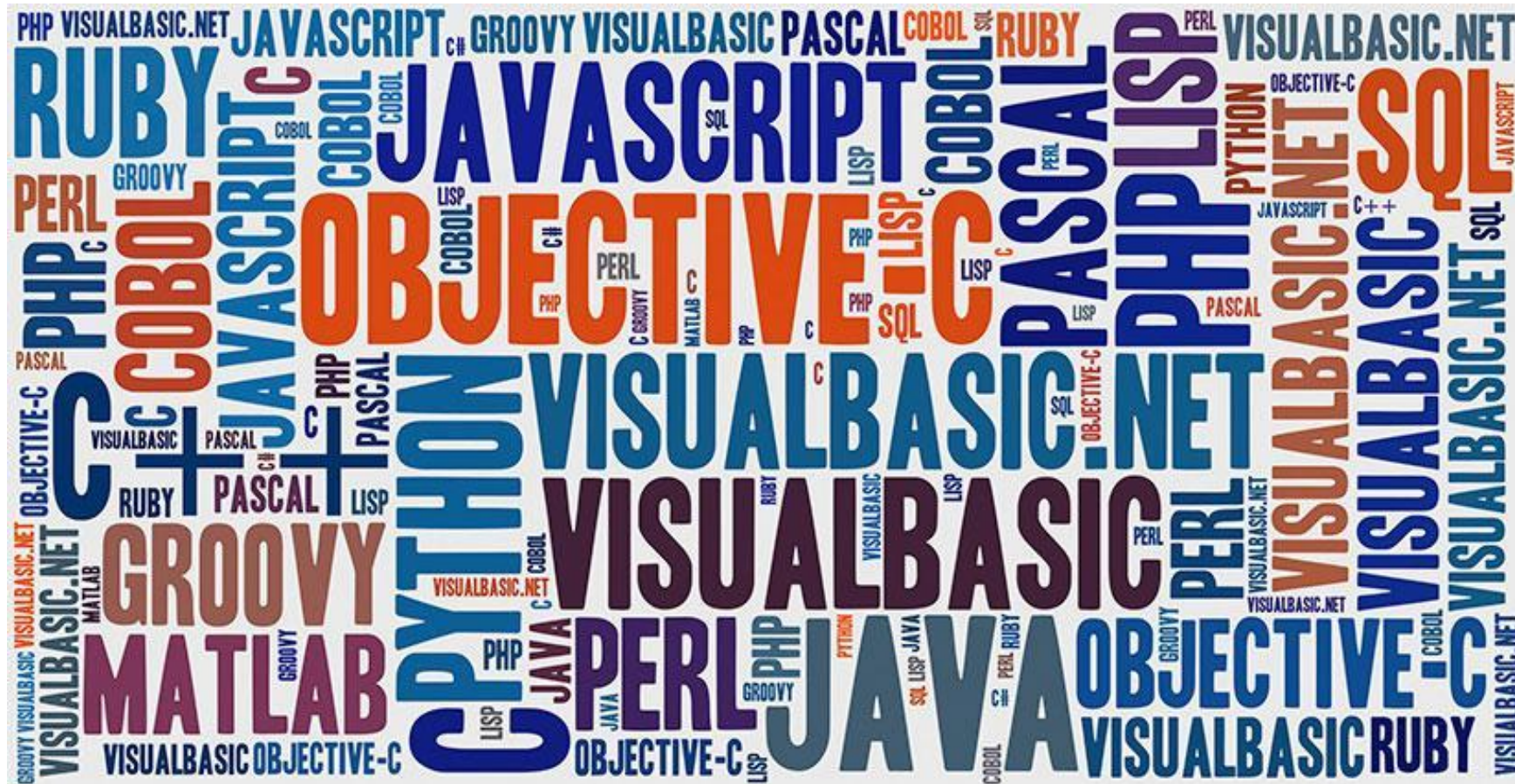






ARTIFICIAL
INTELLIGENCE

Programming Languages



Programming languages

- Programming languages are formal languages used to write computer programs.
- Programming languages are formal languages that have been designed to express computations.

Natural Languages are languages people speak such as English, Amharic, and French

Formal languages are languages people designed for a specific purpose. For example: Mathematical Notations, Chemical Symbols, and Programming languages

Natural vs Formal language

- Formal languages have strict syntax rules to convey a precise message to the intended audience.
- For eg. The chemical symbol for water can only be written as H₂O.
The sum of the numbers 2 and 3 is described as: $2 + 3 = 5$

Natural vs Formal language

The difference between natural and formal languages can be captured:

- **Ambiguity:** natural languages are full of ambiguity and the correspondence relies on contextual clues to parse the meaning of the information being communicated. On the other hand, formal languages are designed to be precise as much as possible.
- **Redundancy:** natural languages rely on redundancy to communicate the message without misunderstanding and ambiguity. Formal languages are less redundant.
- **Literalness:** natural languages are full of metaphors and idioms. Formal languages mean exactly what they say.

Programming languages

Programming languages have syntax and semantics

- The syntax of a program refers to the set of rules that define the structure and format of the program's code.
- The semantics of a program refers to the meaning or interpretation of the program's code.

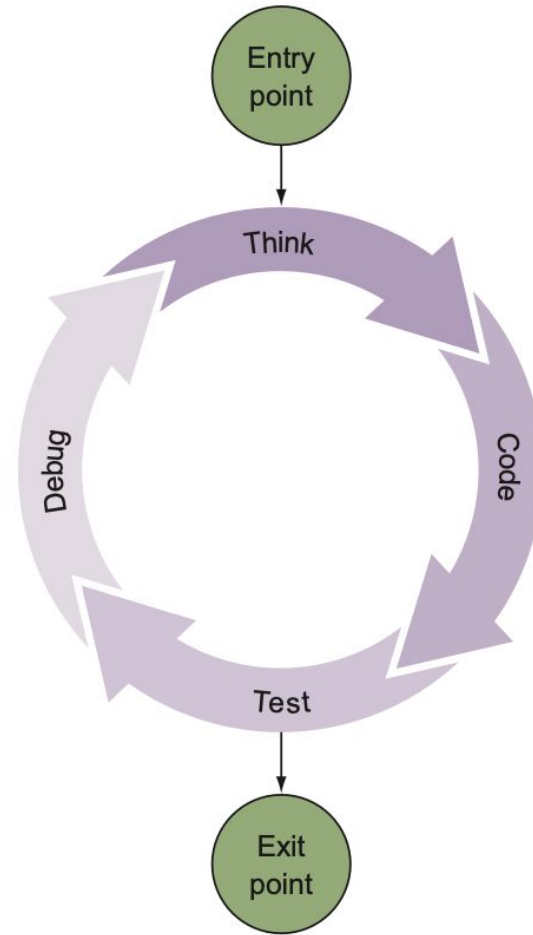
Programming languages

- **High-level Languages:** programming languages that can easily be understood by humans. The languages are almost English like. Examples: Python, C, C++, Java and so on.
- **Low-level languages (Machine language):** language that can be understood by computers and difficult to read by humans. All of the language is written by symbols 0 & 1.

Programming languages

- High-level languages need to be translated to low-level languages to be executed by a computer. To do this we use:
 - **Compilers:** translates the entire program to a low-level language which then can be executed by computers. Eg. Java, C, C++
 - **Interpreters:** translates and executes the code line by line, translating each line to machine code as it is executing. Eg. Python, Javascript, PHP, Ruby

Approaches to Programming



This is the ideal way to approach solving a problem with programming. Understand the problem before you write any code. Then test the code that you write and debug it as necessary. This process repeats until your code passes all tests.

Programming

- The basic concepts you'll learn that can apply to any programming language are as follows
 - Using **variables**, expressions, and statements in programming
 - Getting the program to make decisions based on **conditions**
 - Getting the program to automatically **repeat** tasks under some conditions
 - **Reusing** operations built into the language to help you be more efficient
 - Making your code more readable and easy to maintain by **breaking** a larger task into smaller ones
 - Knowing which **data structure** (a structure already created that can store information in a certain format) is appropriate to use in different situations

Python: Overview

- Python is a high-level, general-purpose programming language known for its simplicity and readability.
- It is widely used across various domains due to its versatility and ease of learning.
- Key Features
 - **Simple and Readable Syntax:** Python's syntax is straightforward, making it easier to learn and use compared to many other programming languages.
 - **Interpreted:** Python code is executed line by line, which makes it easy to debug and test.
 - **Dynamically Typed:** Python does not require explicit declaration of variable types; the type is determined at runtime.
 - **Extensive Libraries:** Python comes with a rich standard library and supports numerous third-party libraries for data manipulation, machine learning, web development, and more.
 - **Cross-Platform:** Python code can run on different operating systems (Windows, MacOS, Linux) without modification.
 - **Open-Source:** Python is free to use and has a large community that contributes to its development and maintenance.

Python: Applications

- Python is used across multiple areas:
 - **Web Development:** Frameworks like *Django* and *Flask* help build robust web applications.
 - **Data Analysis and Machine Learning:** Libraries like *Pandas*, *NumPy*, *Scikit-learn*, *TensorFlow*, and *PyTorch* make Python a favorite among data scientists.
 - **Automation and Scripting:** Python is widely used for automating repetitive tasks.
 - **Scientific Computing:** Libraries like *SciPy* and *Matplotlib* enable scientific research and engineering tasks.
 - **Game Development:** Python supports game development through libraries like *Pygame*.
 - **Embedded Systems and IoT:** Python is used in devices like *Raspberry Pi*.
 - **Artificial Intelligence (AI) and Natural Language Processing (NLP):** Python powers AI and NLP applications through specialized libraries.

Your first program

```
print("Hello world")
```